



**BHASKARACHARYA NATIONAL INSTITUTE FOR SPACE
APPLICATIONS AND GEO-INFORMATICS**

WEEKLY PROGRESS REPORT (05/02/2026 – 11/02/2026)

WEEK 5

PROJECT NAME	Monitoring and change detection system using OSINT
---------------------	---

PROJECT DESCRIPTION :

**CREATING OSINT TOOL FOR MONITORING AND
CHANGE DETECTION SYSTEM BY MONITORING
PUBLICLY AVAILABLE INFORMATION.**

GROUP MEMBER :

Harshil Vaniya, Shalomo Agarwarka, Vrushank Thakkar

GROUP ID :

2026G195

GROUP GUIDE :

CHASIYA KRUTIKA

GROUP COORDINATOR :

SIDHDHARTH PATEL

COLLEGE GUIDE NAME :

BHUMIKA DOSHI

COLLEGE GUIDE No. :

9016888925

COLLEGE GUIDE EMAIL. :

BHUMIKASDOSHI@GMAIL.COM

COLLEGE NAME :

**DEPT. OF BIOCHEMISTRY AND FORENSICS SCIENCE, GUJARAT
UNIVERSITY**

05/02/2026	• Orchestrator compatibility issues solving: subdomain finder, saver and html finder & saver problems (Harshil Vaniya) • Change detection testing and error solving. (Vrushank Thakkar) • Error solving and orchestrator correction. (Shalomo Agarwarkar)
06/02/2026	• Created code that integrates wapiti into current tool using wapiti only. Running but still not getting results up to the mark (Vrushank Thakkar) • Improved change detection code integration & added severity scoring for changes (Harshil Vaniya) • Made a basic GUI for air traffic tracking. (Shalomo Agarwarkar)
07/02/2026	• Added timeline report and started adding trends (Harshil Vaniya) • Tried to fix the errors of wapiti module. Still not upto the mark but better than yesterday will better in upcoming days. (Vrushank Thakkar) • Modified tool to support more flights based on call id and track them by integrating google maps. Has a 5 second refresh timer and a retry function. (Shalomo Agarwarkar)
08/02/2026	Holiday (Sunday)
09/02/2026	• Enhanced wapiti module by adding vulnerability name, overall score overall severity etc and also started to create vt check module. (Vrushank Thakkar) • Added trend analysis and solved some errors. (Harshil Vaniya) • Tried to implement execution of python scripts in browser. Build a py based browser for understanding concepts better. (Shalomo Agarwarkar)
10/02/2026	• Added and improved alert & threshold capabilities. (Harshil Vaniya) • Enhanced vt module by adding a feature that gives total subdomains scanned, malicious found, not malicious. (Vrushank Thakkar) • Installed Ubuntu and configured it with vm. (Shalomo Agarwarkar)

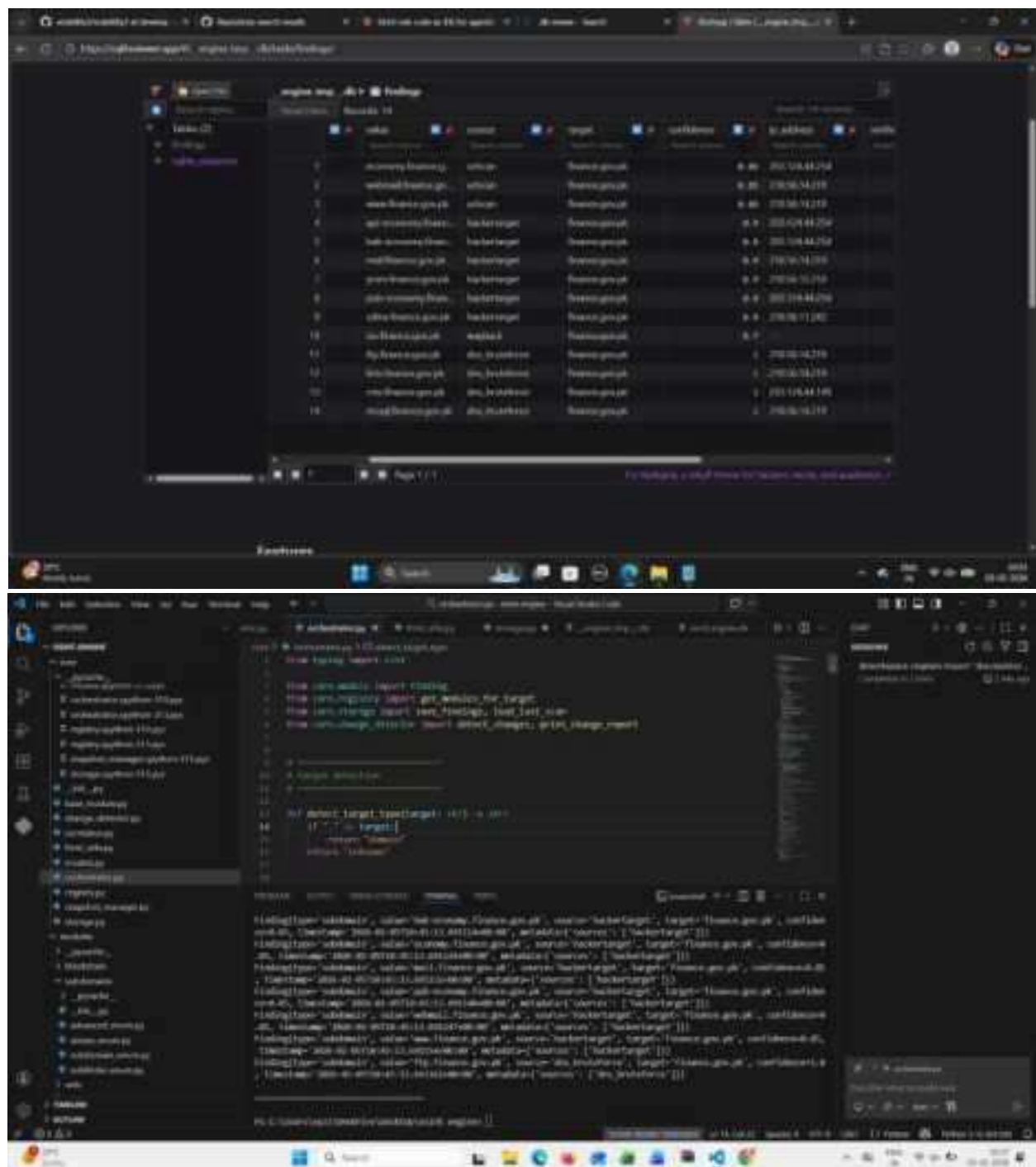
11/02/2026	• Added wayback module to framework (Vaniya Harshil) • Enhanced downloading capabilities of python-based browser and added extension loading capabilities. (Shalomo Agarwarkar) • Created medium OSINT PII detecttool (Vrushank Thakkar)
------------	--

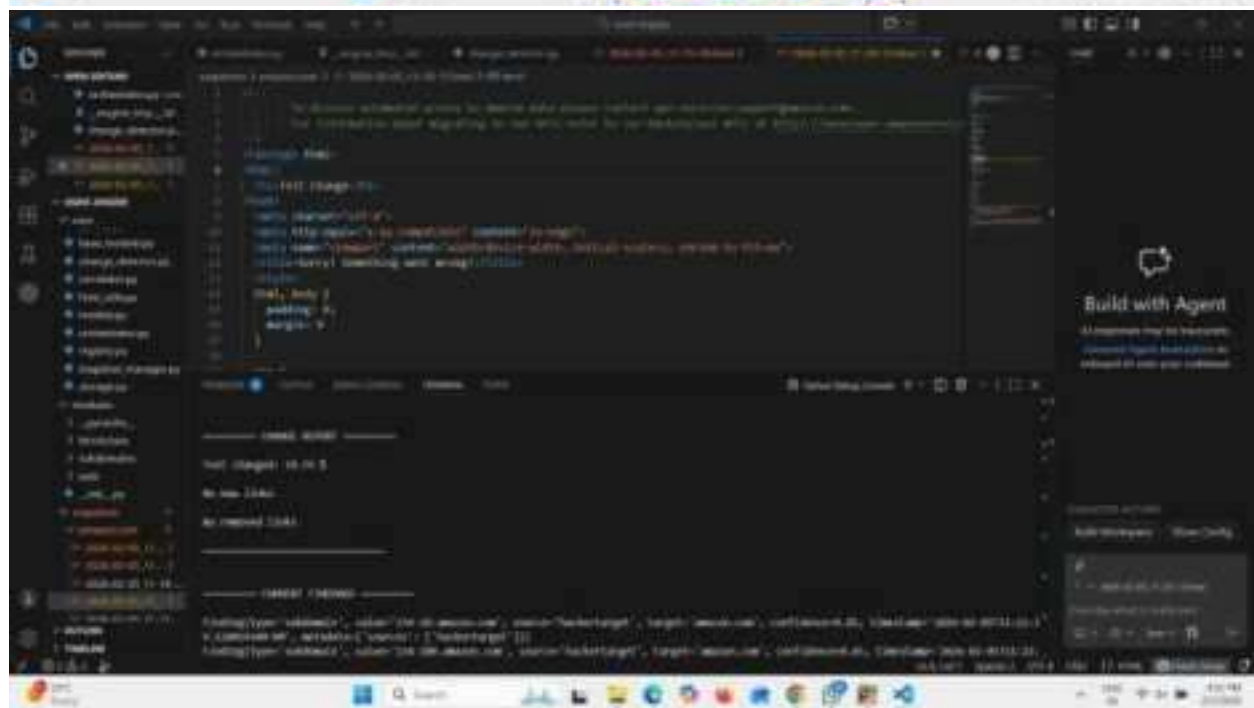
Next Week Plan	Integrating modules into the basic framework of tool
----------------	--

REFERENCE:

- <https://code.visualstudio.com/docs/supporting/terminal-launch>
- <https://docs.python.org/3/library/pathlib.html>
- <https://docs.python.org/3/library/email.examples.html>
- <https://www.geeksforgeeks.org/python/e-mails-notification-bot-with-python/>

SCREEN SHOTS:





```

1 import numpy as np
2 import pandas as pd
3 from sklearn.metrics import mean_squared_error, mean_absolute_error, mean_absolute_percentage_error
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.model_selection import train_test_split
6 from sklearn.linear_model import LinearRegression
7 from sklearn.metrics import r2_score
8
9 # Load the data
10 data = pd.read_csv('data.csv')
11
12 # Split the data into training and testing sets
13 X_train, X_test, y_train, y_test = train_test_split(data[['feature1', 'feature2', 'feature3', 'feature4', 'feature5', 'feature6', 'feature7', 'feature8', 'feature9', 'feature10', 'feature11', 'feature12', 'feature13', 'feature14', 'feature15', 'feature16', 'feature17', 'feature18', 'feature19', 'feature20', 'feature21', 'feature22', 'feature23', 'feature24', 'feature25', 'feature26', 'feature27', 'feature28', 'feature29', 'feature30', 'feature31', 'feature32', 'feature33', 'feature34', 'feature35', 'feature36', 'feature37', 'feature38', 'feature39', 'feature40', 'feature41', 'feature42', 'feature43', 'feature44', 'feature45', 'feature46', 'feature47', 'feature48', 'feature49', 'feature50', 'feature51', 'feature52', 'feature53', 'feature54', 'feature55', 'feature56', 'feature57', 'feature58', 'feature59', 'feature60', 'feature61', 'feature62', 'feature63', 'feature64', 'feature65', 'feature66', 'feature67', 'feature68', 'feature69', 'feature70', 'feature71', 'feature72', 'feature73', 'feature74', 'feature75', 'feature76', 'feature77', 'feature78', 'feature79', 'feature80', 'feature81', 'feature82', 'feature83', 'feature84', 'feature85', 'feature86', 'feature87', 'feature88', 'feature89', 'feature90', 'feature91', 'feature92', 'feature93', 'feature94', 'feature95', 'feature96', 'feature97', 'feature98', 'feature99', 'feature100'], data[['target']], test_size=0.2, random_state=42)
14
15 # Standardize the features
16 scaler = StandardScaler()
17 X_train = scaler.fit_transform(X_train)
18 X_test = scaler.transform(X_test)
19
20 # Train the Linear Regression model
21 model = LinearRegression()
22 model.fit(X_train, y_train)
23
24 # Evaluate the model
25 y_pred = model.predict(X_test)
26 mse = mean_squared_error(y_test, y_pred)
27 mae = mean_absolute_error(y_test, y_pred)
28 mape = mean_absolute_percentage_error(y_test, y_pred)
29 r2 = r2_score(y_test, y_pred)
30
31 # Print the results
32 print('MSE: ', mse)
33 print('MAE: ', mae)
34 print('MAPE: ', mape)
35 print('R2: ', r2)

```

```

1 # Feature Engineering
2
3 # Create a new feature based on the existing features
4 data['feature36'] = data['feature1'] * data['feature2']
5
6 # Split the data into training and testing sets
7 X_train, X_test, y_train, y_test = train_test_split(data[['feature1', 'feature2', 'feature3', 'feature4', 'feature5', 'feature6', 'feature7', 'feature8', 'feature9', 'feature10', 'feature11', 'feature12', 'feature13', 'feature14', 'feature15', 'feature16', 'feature17', 'feature18', 'feature19', 'feature20', 'feature21', 'feature22', 'feature23', 'feature24', 'feature25', 'feature26', 'feature27', 'feature28', 'feature29', 'feature30', 'feature31', 'feature32', 'feature33', 'feature34', 'feature35', 'feature36', 'feature37', 'feature38', 'feature39', 'feature40', 'feature41', 'feature42', 'feature43', 'feature44', 'feature45', 'feature46', 'feature47', 'feature48', 'feature49', 'feature50', 'feature51', 'feature52', 'feature53', 'feature54', 'feature55', 'feature56', 'feature57', 'feature58', 'feature59', 'feature60', 'feature61', 'feature62', 'feature63', 'feature64', 'feature65', 'feature66', 'feature67', 'feature68', 'feature69', 'feature70', 'feature71', 'feature72', 'feature73', 'feature74', 'feature75', 'feature76', 'feature77', 'feature78', 'feature79', 'feature80', 'feature81', 'feature82', 'feature83', 'feature84', 'feature85', 'feature86', 'feature87', 'feature88', 'feature89', 'feature90', 'feature91', 'feature92', 'feature93', 'feature94', 'feature95', 'feature96', 'feature97', 'feature98', 'feature99', 'feature100'], data[['target']], test_size=0.2, random_state=42)
8
9 # Standardize the features
10 scaler = StandardScaler()
11 X_train = scaler.fit_transform(X_train)
12 X_test = scaler.transform(X_test)
13
14 # Train the Linear Regression model
15 model = LinearRegression()
16 model.fit(X_train, y_train)
17
18 # Evaluate the model
19 y_pred = model.predict(X_test)
20 mse = mean_squared_error(y_test, y_pred)
21 mae = mean_absolute_error(y_test, y_pred)
22 mape = mean_absolute_percentage_error(y_test, y_pred)
23 r2 = r2_score(y_test, y_pred)
24
25 # Print the results
26 print('MSE: ', mse)
27 print('MAE: ', mae)
28 print('MAPE: ', mape)
29 print('R2: ', r2)

```